DESIGN DOCUMENT

Parallel Redesign of a Cycle-Accurate VHDL Event-Driven Simulator

Course	COD7001- Cornerstone Project
Assignment	Parallel Discrete-Event Simulator
Team Members	Chirag Suthar (2025MCS2105) | Shreshth Verma (2025MCS2094)
Document Version	v1.0 — Initial Design Document
Repository	github.com/shreshth23/cornerstone-VHDL-simulator-parallelized

Submitted to: Prof. Kolin Paul

1. Introduction and Problem Statement

Event-driven simulation is the foundation of all modern hardware description language (HDL) toolchains. Before a chip is fabricated, designers simulate its behavior in software — and that simulation must be both cycle-accurate and semantically identical to what the actual hardware will produce. VHDL simulators accomplish this by maintaining a global event queue ordered by logical simulation time and processing each event strictly in sequence.

While this sequential approach is provably correct, it is deeply inefficient on modern hardware. A contemporary workstation may have 16 or more CPU cores, yet a traditional VHDL simulator uses only one. The goal of this project is to redesign the simulation kernel to exploit this idle parallelism — while producing bit-for-bit identical output to the sequential baseline.

1.1 Why Naive Parallelization Fails

The fundamental challenge is that logical simulation time is not physical time. Consider two events scheduled at the same timestamp $t = 10\,ns$. Naively, one might run them on two threads simultaneously. But if event B reads a signal that event A writes, running them concurrently produces a race condition and incorrect simulation output — with no crash or error message, just wrong waveforms.

A further complication is VHDL's delta cycle semantics. Within a single simulation timestep, signals may propagate through multiple stages of combinational logic — each stage separated by a delta cycle that has zero physical time but strict causal ordering. Violating this order is equivalent to reading a wire before its driver has settled, which is fundamentally incorrect behavior.

> **Core Insight:** Safe parallelism exists only between processes that share no signal dependencies within the same timestep and delta cycle. Identifying this independence is the central technical challenge of this project.

1.2 Project Scope

This document presents the complete design for our parallel VHDL simulator, covering: the sequential baseline architecture, the dependency analysis framework, the chosen parallelization strategy, synchronization mechanisms, correctness verification methodology, and projected performance results. Implementation proceeds in phases over 80 days as specified in the assignment brief.

2. System Architecture Overview

Our simulator is organized into four major subsystems that operate as a pipeline during simulation. Each subsystem is independently testable and uses the same interfaces in both the sequential baseline and the parallel version — a deliberate design choice that simplifies correctness verification.

2.1 Subsystems

- **Event Queue Manager** — Maintains the global priority queue ordered by `(timestamp, delta_cycle)`. Thread-safe in the parallel version using fine-grained locking. Accessed only by the dispatcher thread to eliminate contention.
- **Process Execution Engine** — Executes VHDL processes when their sensitivity list is triggered. In the parallel version, independent processes execute concurrently on a persistent thread pool.
- **Signal State Store** — Holds current and next-cycle values of every signal. Implements VHDL's two-phase update protocol (schedule then commit) with atomic double-buffering.
- **Waveform Logger(optional)** — Records signal transitions for correctness verification and VCD output. Writes are buffered per-thread and flushed deterministically at timestep boundaries.

2.2 Sequential vs. Parallel Loop

The sequential baseline processes one event at a time. The parallel version adds a dependency-aware batch dispatcher between event dequeuing and process execution:

Sequential Baseline

```
while queue not empty:
  event = queue.pop()
  execute(event)
  schedule_new_events()
  // one at a time
```

Parallel Version

```
while queue not empty:
  batch = collect_same_ts()
  groups = dep_partition(batch)
  parallel_execute(groups)
  barrier(); commit_signals()
```

The key additions in the parallel version are batch collection, dependency partitioning, and barrier-synchronized signal commitment. These three steps together guarantee that VHDL semantics are preserved exactly.

3. Dependency Analysis

Before parallel execution can take place, the simulator must determine which processes can safely run simultaneously. Two processes are independent if and only if neither reads a signal the other writes, within the same timestep and delta cycle.

3.1 Dependency Graph Construction

We construct a static dependency graph $G = (V, E)$ during initialization, where each vertex is a VHDL process. A directed edge $P_1 \to P_2$ exists if P_1 writes to any signal that P_2 is sensitive to. For each process P, we track two sets: $\text{Reads}(P)$ (signals read) and $\text{Writes}(P)$ (signals driven). An edge $P_1 \to P_2$ is added whenever $\text{Writes}(P_1) \cap \text{Reads}(P_2) \neq \emptyset$.

> **Design Choice:** We use static dependency analysis (conservative over-approximation). We may occasionally serialize processes that are independent at runtime, but we will never incorrectly parallelize dependent ones. Correctness is guaranteed; a small amount of performance is traded for safety.

3.2 Independent Set Extraction

From the dependency graph, we extract independent process sets using greedy graph coloring. Processes assigned the same color have no dependency edges between them and can execute in the same parallel wave. This coloring is computed once at startup and reused every timestep, since signal dependencies in RTL designs are static.

1. **Step 1:** Parse all process sensitivity lists and signal assignments.
2. **Step 2:** Build Read/Write sets for each process.

3. **Step 3:** Construct the directed dependency graph.
4. **Step 4:** Compute independent sets via greedy graph coloring.
5. **Step 5:** Cache the partitioning for reuse during simulation.

3.3 Delta Cycle Handling

Delta cycles add a layer of ordering within a single timestamp. Process A may write signal S at time T with delta 0, triggering process B at time T with delta 1. These two cannot execute in parallel even though they share the same physical timestamp. Our design handles this by treating `(timestamp, delta_cycle)` as a compound key. Events are grouped and compared by this compound key, not just timestamp. This preserves VHDL's delta-cycle semantics exactly.

4. Parallelization Strategy

We evaluated all four parallelization strategies suggested in the assignment before selecting our approach:

Strategy	Parallelism Grain
Time-slice parallelism	Events at same timestamp
Delta-cycle parallelism	Independent delta cycles
Partitioned event queues	Per-partition events
Speculative + rollback	All events speculatively

4.1 Chosen Strategy: Time-Slice Parallelism with Delta-Cycle Barriers

We adopt time-slice parallelism as our primary strategy, enhanced with delta-cycle barriers. At each simulation step, all events sharing the current `(timestamp, delta_cycle)` are dequeued as a batch. The dependency partitioner assigns each event to an independent group. A persistent thread pool executes one wave at a time, with all processes within a wave running truly in parallel. After all waves in a delta cycle complete, a global barrier enforces signal commitment before the next `(timestamp, delta_cycle)` begins.

4.2 Thread Pool Design

We implement a persistent thread pool using C++ `std::thread`, with a fallback OpenMP path for benchmarking comparison. Creating and destroying threads per-event would introduce overhead that exceeds any parallelism benefit. Worker threads are created once at startup, block on a condition variable, and are woken in batches when work is available. Thread count defaults to `std::thread::hardware_concurrency()` but is configurable.

- **Process-to-thread assignment:** round-robin by process ID (deterministic).
- **Work distribution:** per-group task lists dispatched when wave begins.
- **Synchronization:** `std::barrier` (C++20) at wave boundaries; fallback `pthread_barrier`.
- **Signal commits:** deferred to a single commit step after the barrier, by one designated thread.

4.3 Why Not Speculative Execution

Speculative execution with rollback was considered and rejected. Checkpointing the entire signal state at each event is expensive, and rollback frequency for typical RTL designs with dense signal dependencies would likely make overhead worse than the sequential baseline. This remains a potential future enhancement for sparse, pipeline-dominated designs.

5. Synchronization and Determinism

Determinism is a hard requirement: the parallel simulator must produce identical output on every run, regardless of OS thread scheduling. We achieve this through barrier synchronization, atomic double-buffered signal updates, and deterministic event ordering.

5.1 Barrier Placement

Barriers are placed at exactly two points per delta cycle. The first barrier fires after all processes in a parallel wave complete — preventing any process from reading a signal that another process in the same wave is still writing. The second barrier fires after signal commitment — ensuring all signals are in their new state before the next delta cycle begins. No process may proceed past a barrier until all threads have reached it.

Key Invariant: No thread may read a signal's committed value until all writers in the same delta cycle have finished and the commit barrier has been crossed. This invariant is enforced by construction and is the foundation of our correctness guarantee.

5.2 Two-Phase Signal Update Protocol

Signal updates in VHDL are inherently two-phase: a process schedules a new value, but that value only becomes visible after the current delta cycle ends. We model this with a double-buffer per signal: a "current" value (read-only during execution) and a "next" value (write-only during execution). At the delta-cycle boundary, one designated thread promotes all next-values to current-values while all other threads wait at the barrier.

5.3 Reproducibility Guarantee

Process-to-thread assignment is fixed and deterministic (`process_id mod num_threads`). Signal commits happen after a global barrier, not as individual threads finish. New event enqueuing is performed by a single dispatcher thread that merges per-thread pending lists in a fixed order. Together, these three design decisions ensure that the simulation trace is identical on every run, even under different OS scheduling decisions.

5.4 Event Queue Access Pattern

The global event queue is accessed exclusively by the main dispatcher thread. Worker threads never enqueue events directly — instead, they write new events to a thread-local pending list during execution. After each wave barrier, the dispatcher thread merges all pending lists into the global queue in deterministic order. This design eliminates all contention on the event queue.

6. Correctness Verification

The assignment requires bit-for-bit identical waveform output between the sequential and parallel simulators. We use two complementary verification techniques throughout development.

6.1 VCD Waveform Comparison

Both simulators produce Value Change Dump (VCD) files — the standard format for VHDL simulation waveforms. Our test harness runs both simulators on identical inputs, then performs a line-by-line diff of their VCD outputs. Any discrepancy at any timestamp or signal name is treated as a blocking correctness failure. This is the primary verification method during development.

6.2 Trace Hashing

For automated regression testing, we compute a SHA-256 hash of each simulator's waveform trace, serialized as a sorted sequence of `(time, signal_name, value)` tuples. The sequential simulator's hash becomes the ground truth. The parallel simulator must produce an identical hash. This allows thousands of randomized tests to run efficiently without storing full VCD files for each.

6.3 Test Circuit Suite

Correctness is verified on a progressive suite of circuits, ordered by complexity:

- Simple combinational gates (AND, OR, XOR) — verifies basic event handling.
- D flip-flop and register file — verifies clocked sequential logic and delta cycles.

- 4-bit ripple carry adder — verifies multi-process dependency chains.
- 4-state finite state machine — verifies feedback loops and state-dependent transitions.
- 5-stage in-order pipeline — verifies inter-stage data forwarding and stall logic.
- Bus arbitration circuit — verifies shared-resource access and priority ordering.

Each circuit is tested with at least 1,000 randomized input vectors. Any waveform mismatch between sequential and parallel outputs is treated as a blocking bug and must be resolved before advancing to the next complexity tier.

7. Performance Evaluation Plan

Performance is measured on two axes: speedup relative to the sequential baseline, and scalability as core count increases. The projections below are derived from Amdahl's Law, using an estimated parallelizable fraction of 0.65 based on typical RTL design dependency density. These will be validated against actual benchmark results as implementation progresses.

- What you do for Complete VITDL
- The parallelization strategy is Unclear to me
- also why SIMD Cannot be used?

6.5/7.5